# Unified Team KPIs 2026

🏠 Dashboard 📚 Resources

# Unified Team KPIs 2026

## Purpose

To measure the **health, predictability, and quality of the entire delivery system** as a unified team. These metrics focus on outcomes, system behaviour, and flow, rather than silencing specific roles (Engineering, Product, QA).

The goal is to ensure that **Scope**, **Code**, and **Validation** work together to deliver value predictably.

> **Status Flow Reference:**
>
> `New` → `Ready` → `Active` → `Ready For QA` → `In QA` → `Resolved` → `Deployed`

---

# 1. System Quality & Stability

*Focus: Is the work we deliver correct, robust, and safe?*

## 1.1 Rework Rate ⚠️

### What it measures

How often work is incorrect or incomplete when first delivered to QA, causing churn in the flow.

### Definition

Percentage of items that move backwards from `Ready For QA` (or `In QA`) to `Active` due to implementation defects.

### How to measure

Only count when:

- Item moves from `Ready For QA` / `In QA` → `Active`
- Reason / Root Cause = `Code Defect`

**Formula**

```
rework_rate = items_returned_to_active / total_items_reachi
ng_ready_for_qa
```

### Real example

- 50 items reached `Ready For QA`
- 5 items moved back to `Active` for fixes

```
rework_rate = 5 / 50 = 10%
```

### RAG Thresholds

- 🟢 Green: **≤ 10%**
- 🟠 Amber: **10–15%**
- 🔴 Red: **> 15%**

## 1.2 Scope Quality ⚠️

### What it measures

How well work is understood and defined before development starts, avoiding waste due to ambiguity.

### Definition

Percentage of items that require rework or move backwards due to **missing or misunderstood scope/requirements**.

### How to measure

Only count when:

- Item moves backwards (e.g., `Active` → `Ready` or `New`, or stays `Active` with blocked time)
- Root Cause = `Scope / Requirements`

#### Formula

```
scope_rework_rate = items_reopened_due_to_scope / total_com
pleted_items
```

### Real example

- 40 items deployed
- 4 had to be redefined or clarified significantly during `Active` state

```
scope_rework_rate = 4 / 40 = 10%
```

### RAG Thresholds

- 🟢 Green: **≤ 10%**
- 🟠 Amber: **10–15%**
- 🔴 Red: **> 15%**

## 1.3 Defect Leakage & Escaped Defects 🚨

### What it measures

The effectiveness of the quality net (QA + Automation) in preventing defects from reaching  Production.

### Definition

1. **Escaped Defects:** Critical (P1/P2) defects found in prod environment within 14 days.
2. **Defect Leakage:** Ratio of defects found in pro vs. stage.

### How to measure

#### Formula (Leakage)

```
leakage = defects_found_in_prod / total_defects_stage
```

### Real example

- 100 total defects found (95 in `In QA`, 5 in prod)

```
leakage = 5 / 100 = 5%
```

### RAG Thresholds

**Escaped Defects (Count/Quarter)**

- 🟢 Green: **0–1**
- 🟠 Amber: **2–3**
- 🔴 Red: **> 3**

**Leakage (%)**

- 🟢 Green: **≤ 3%**
- 🟠 Amber: **3–6%**
- 🔴 Red: **> 6%**

## 1.4 Automation Coverage %

### What it measures

How much of the product's behaviour is protected by automated tests to ensure safe deployments to `Resolved` and `Deployed`.

### Definition

The proportion of relevant testable scenarios covered by automated tests.

### How to measure

**Formula**

```
automation_coverage = automated_test_cases / total_test_cas
es
```

### Real example

- 200 test cases total
- 150 automated

```
automation_coverage = 150 / 200 = 75%
```

### RAG Thresholds

- 🟢 Green: **≥ 75%**
- 🟠 Amber: **60–75%**
- 🔴 Red: **< 60%**

## 1.5 Definition of Done (DoD) Adherence ✅

### What it measures

Whether items consistently meet all quality criteria before moving to `Resolved`.

### Definition

Percentage of items in `Resolved` that satisfy **all mandatory DoD criteria** (e.g., Code Reviewed, Unit Tests Passed, QA Signed off).

### How to measure

#### Formula

```
dod_adherence = items_meeting_all_criteria / total_items_re
solved
```

### Real example

- 50 items reached `Resolved`
- 48 had all DoD checklists complete

```
dod_adherence = 48 / 50 = 96%
```

### RAG Thresholds

- 🟢 Green: **≥ 95%**
- 🟠 Amber: **90–95%**
- 🔴 Red: **< 90%**

---

# 2. Flow & Delivery Health

*Focus: Does work flow smoothly and predictably through the system?*

## 2.1 Delivery Predictability 📊

### What it measures

How reliably the team delivers the work they commit to.

### Definition

Ratio of items that reach `Deployed` (or `Resolved`) compared to items that were committed to `Active` (or `Ready`) at the start of the period.

### How to measure

#### Formula

```
delivery_predictability = items_deployed / items_committed
```

### Real example

- Team committed 20 items (moved to `Active` at sprint start)
- 18 items reached `Deployed`

```
delivery_predictability = 18 / 20 = 90%
```

### RAG Thresholds

- 🟢 Green: **85–100%**
- 🟠 Amber: **70–85%**
- 🔴 Red: **< 70%**

## 2.2 WIP Discipline

### What it measures

Whether the team limits concurrent work in `Active`, `Ready For QA`, and `In QA` to match capacity.

### Definition

The percentage of time the team operates within agreed WIP limits for `Active` and `In QA` states.

### How to measure

#### Formula

```
wip_compliance = time(wip_level ≤ limit) / total_time
```

### Real example

- Week has 40 hours
- Team stayed under WIP limit for 36 hours

```
wip_compliance = 36 / 40 = 90%
```

### RAG Thresholds

- 🟢 Green: **≥ 90%**
- 🟠 Amber: **80–90%**
- 🔴 Red: **< 80%**

## 2.3 Flow Hygiene

### What it measures

Whether work flows smoothly without bottling up in wait states like `Ready For QA`.

### Definition

The ratio of items queuing in a specific status (e.g., `Ready For QA`) against the limit.

### How to measure

#### Formula

```
queue_load = avg_items_in_ready_for_qa / limit_for_ready_fo
```

```
r_qa
```

### Real example

- Limit for `Ready For QA`: 3 items
- Average items observed: 4

```
queue_load = 4 / 3 = 1.33 (Bottleneck)
```

### RAG Thresholds

- 🟢 Green: **≤ 1.0**
- 🟠 Amber: **1.0–1.2**
- 🔴 Red: **> 1.2**

## 2.4 Speed of Automating New Features

### What it measures

How quickly automation catches up to new functionality.

### Definition

Time elapsed between an item reaching `Ready For QA` (or `Resolved`) and the corresponding automated tests being completed.

### How to measure

#### Formula

```
automation_lag = date_automation_done - date_item_resolved
```

### Real example

- Item reached `Resolved`: Jan 1st
- Automation committed: Jan 5th

```
lag = 4 days
```

### RAG Thresholds

- 🟢 Green: **≤ 7 days**
- 🟠 Amber: **7–14 days**
- 🔴 Red: **> 14 days**

---

# 3. Strategic Investment & Alignment

*Focus: Are we working on the right things and sustaining the platform?*

## 3.1 Delivery of Specific Initiatives 🎯

### What it measures

Whether agreed strategic initiatives (Epics) are actually completed.

### Definition

Ratio of completed strategic initiatives to those agreed for the period.

### How to measure

**Formula**

```
initiative_delivery = completed_initiatives / agreed_initia
tives
```

### Real example

- 5 Initiatives planned
- 4 Completed (`Deployed`)

```
Result = 80%
```

### RAG Thresholds

- 🟢 Green: **≥ 80%**
- 🟠 Amber: **60–80%**
- 🔴 Red: **< 60%**

## 3.2 Technical Debt Investment Ratio 🧱

### What it measures

How much delivery capacity is invested in paying down technical debt vs. new features.

### Definition

Percentage of items in `Deployed` that were classified as **Technical Debt**.

### How to measure

#### Formula

```
tech_debt_ratio = tech_debt_items_deployed / total_items_de
ployed
```

### Real example

- 50 items `Deployed`
- 10 were Tech Debt

```
Ratio = 20%
```

### RAG Thresholds

- 🟢 Green: **20–30%**
- 🟠 Amber: **10–20%** or **> 30%**
- 🔴 Red: **< 10%**

---

# 4. Operational Excellence & Reliability Culture

*Focus: Are we learning from failure and building a culture of preparedness?*

## 4.1 Post-Mortem Discipline 🧠

### What it measures

Whether we consistently learn from critical failures.

### Definition

Percentage of **P0/P1 incidents** that have a completed post-mortem document.

### How to measure

#### Formula

```
pm_completion = completed_post_mortems / total_p0_p1_incide
nts
```

### Real example

- 4 P1 incidents this quarter
- 4 Post-mortems completed

```
Result = 100%
```

### RAG Thresholds

- 🟢 Green: **100%**
- 🔴 Red: **< 100%** (Every critical incident requires learning)

## 4.2 Reliability Action Delivery 🛡️

### What it measures

Whether we actually fix the root causes we identify, rather than just talking about them.

### Definition

Percentage of **priority post-mortem actions** delivered within the agreed SLO (e.g., 8 weeks).

### How to measure

#### Formula

```
action_delivery = actions_completed_in_slo / total_identifi
ed_actions
```

### Real example

- 10 priority actions identified
- 8 completed within agreed SLO

```
Result = 80%
```

### RAG Thresholds

- 🟢 Green: **≥ 80%**
- 🟠 Amber: **60–80%**
- 🔴 Red: **< 60%**

## 4.3 Recurring Incident Reduction 📉

### What it measures

Effectiveness of our fixes. We should not fail in the same way twice.

### Definition

Percentage reduction in **repeated incidents** (incidents with the same root cause) Quarter-over-Quarter.

### How to measure

#### Formula

```
recurrence_reduction = (prev_q_repeated - curr_q_repeated)
/ prev_q_repeated
```

### Real example

- Q1: 5 repeated incidents
- Q2: 2 repeated incidents

```
Reduction = 60%
```

### RAG Thresholds

- 🟢 Green: **> 0% reduction** (Improving)
- 🔴 Red: **≤ 0%** (Stagnant or worsening)

## 4.4 Playbook Coverage 📖

### What it measures

Operational readiness. Can on-call engineers handle critical systems without guessing?

### Definition

Percentage of **critical implementations/services** that have an up-to-date runbook/playbook.

### How to measure

#### Formula

```
playbook_coverage = services_with_playbooks / total_critica
l_services
```

### Real example

- 10 critical services
- 9 have playbooks

```
Result = 90%
```

### RAG Thresholds

- 🟢 Green: **100%**
- 🟠 Amber: **90–99%**
- 🔴 Red: **< 90%**

## 4.5 Knowledge Sharing (Demos) 📺

### What it measures

Ensuring features are understood by the wider team and stakeholders before/at delivery.

### Definition

Percentage of **critical features** delivered that include a recorded demo session or live walkthrough.

### How to measure

**Formula**

```
demo_coverage = features_with_demos / total_critical_featur
es
```

### Real example

- 5 critical features released
- 5 demos conducted

```
Result = 100%
```

### RAG Thresholds

- 🟢 Green: **100%**
- 🟠 Amber: **80–99%**
- 🔴 Red: **< 80%**

---

# 5. Health Checks (DORA Metrics) — Not KPIs 🩺

*Focus: Diagnostic gauges for team health, not performance targets.*

## Purpose

We track DORA metrics to **gauge system friction and operational health**. These are **indicators**, not KPIs.

- **If they trend down:** The team should investigate (e.g., "Is our pipeline too slow?", "Are we batching too much?").
- **Usage:** For team retrospectives and self-improvement, not for management reporting or RAG status.

## The Metrics

1. **Deployment Frequency:** How often we ship to production.
2. **Lead Time for Changes:** Time from code commit to running in production.
3. **Change Failure Rate:** Percentage of deployments causing a failure in production.
4. **Time to Restore Service:** How long it takes to recover from a failure.